LAB4 FPGA/MCU
실험 후 보고서

실험자 : 권진혁
학번 : 2023440007
학과 : 전자전기컴퓨공학부
실험 일시 : 2026. 10. 12.

1. 과제 1 : 버튼을 이용한 LED 점멸 주기 변경

· 설계 목표

STM32 보드의 사용자 버튼 입력을 읽어 LED의 점멸 주기를 변경한다. 버튼을 누를 때마다 LED 점멸 주기가 다음과 같이 순환하도록 구현한다.

'1초 → 2초 → 4초 → 8초 → 1초'

· 설계 방법

1) GPIO 입력으로 USER Button 설정
2) GPIO 출력으로 내장 LED 설정
3) 버튼 입력이 감지되면 점멸 주기 값 변경
4) LED는 현재 설정된 주기에 따라 ON/OFF 반복
5) 마지막 단계(8초) 이후 다시 1초로 복귀

· 핵심 코드

```c
void polling_routine(void)
{
    /* LED 제어용 변수 */
    static uint32_t last_led_ms = 0;
    static GPIO_PinState led_state = GPIO_PIN_RESET;

    /* 버튼 제어용 변수 */
    static GPIO_PinState previous = GPIO_PIN_RESET;
    static uint32_t last_press_ms = 0;

    /* 추가 변수 : LED 간격 */
    static uint32_t interval_ms = 1000;

    uint32_t now_ms = HAL_GetTick();

    /* ----------------------------
       버튼 처리
       ---------------------------- */
    GPIO_PinState current =
        HAL_GPIO_ReadPin(BUTTON_ece2_GPIO_Port,
                         BUTTON_ece2_Pin);

    if ((current == GPIO_PIN_SET) &&
        (previous == GPIO_PIN_RESET) &&
        (now_ms - last_press_ms >= 200))
    {
        last_press_ms = now_ms;

        /* 1s -> 2s -> 4s -> 8s -> 1s */
        if(interval_ms == 1000)
            interval_ms = 2000;
        else if(interval_ms == 2000)
            interval_ms = 4000;
        else if(interval_ms == 4000)
            interval_ms = 8000;
        else
            interval_ms = 1000;
    }

    previous = current;

    /* ----------------------------
       LED 깜빡임 처리
       ---------------------------- */
    if(now_ms - last_led_ms >= interval_ms)
    {
        last_led_ms = now_ms;

        led_state =
            (led_state == GPIO_PIN_RESET) ?
            GPIO_PIN_SET : GPIO_PIN_RESET;

        HAL_GPIO_WritePin(LED_ece2_GPIO_Port,
                          LED_ece2_Pin,
                          led_state);
    }
}
```

· 검증 결과

버튼을 누를 때마다 LED 점멸 간격이 '1초 → 2초 → 4초 → 8초 → 1초' 순으로 정상적으로 변경되는 것을 확인하였다. 또한, 버튼을 계속 누르고 있어도 간격이 한 번만 변경되어 요구 조건을 만족하였다.

· 문제 해결 과정

초기 구현에서는 버튼을 누르고 있는 동안 간격이 계속 변경되는 문제가 발생하였다. 이를 해결하기 위해 이전 버튼 상태를 저장하는 변수를 추가하여 상승 에지만 검출하도록 수정하였다. 또한, 스위치 채터링에 의해 여러 번 입력되는 현상을 방지하기 위하여 200ms 디바운싱을 적용하였다.

2. 과제 2 : 외부 LED 구현

· 설계 목표

과제 1의 동작을 보드 내장 LED 대신 외부 LED에서 구현한다.

· 설계 방법

브레드보드에 LED와 전류 제한 저항(220Ω)을 연결하였고 GPIO 출력 핀을 통해 LED를 구동하도록 하였다. 기존 프로그램에서 LED

핀 정의만 외부 GPIO로 변경하였다.

· **회로 구조**

STM32 → LED → 330Ω → GND

· **핵심 코드**

C

```c
if(now_ms - last_led_ms >= interval_ms)
{
    last_led_ms = now_ms;

    led_state =
        (led_state == GPIO_PIN_RESET)
        ? GPIO_PIN_SET
        : GPIO_PIN_RESET;

    HAL_GPIO_WritePin(GPIOB,
                      GPIO_PIN_0,
                      led_state);
}
```

- 설정된 점멸 간격(interval_ms)이 지나면 LED 상태를 반전
- GPIOB, GPIO_PIN_0은 외부 LED가 연결된 핀
- 과제 1의 내장 LED 대신 외부 LED를 제어하도록 변경

C

```c
if((current == GPIO_PIN_SET) &&
   (previous == GPIO_PIN_RESET) &&
   (now_ms - last_press_ms >= 200))
{
    last_press_ms = now_ms;

    if(interval_ms == 1000)
        interval_ms = 2000;
    else if(interval_ms == 2000)
        interval_ms = 4000;
    else if(interval_ms == 4000)
        interval_ms = 8000;
    else
        interval_ms = 1000;
}
```

- 버튼이 눌린 순간만 인식
- 점멸 간격을 '1초 → 2초 → 4초 → 8초 → 1초' 순으로 변경
- 200ms 디바운싱을 적용하여 채터링을 방지

· **검증 결과**

외부 LED가 내장 LED와 동일하게 '1초 → 2초 → 4초 → 8초' 간격으로 점멸함을 확인하였다. 버튼 입력에 따라 점멸 간격도 정상적으로 변경되었다.

· **문제 해결 과정**

초기 실험에서 LED 극성을 반대로 연결하여 점등되지 않았다. LED의 Anode와 Cathode 방향을 수정한 후 정상 동작을 확인하였다. 또한, 저항을 연결하지 않을 경우 LED에 과전류가 흐를 수 있으므로 반드시 직렬 저항을 사용하였다.

3. FPGA 실습 1 : 4비트 곱셈기

· **설계 목표**

두 개의 4비트 입력값을 곱하여 8비트 결과를 출력하는 조합논리 회로를 FPGA에 구현한다.

· **설계 방법**

- 입력 : A[3:0], B[3:0]
- 출력 : P[7:0]

Verilog

```verilog
module multiplier(
    input [3:0] A,
    input [3:0] B,
    output [7:0] P
);

assign P = A * B;

endmodule
`
```

Verilog의 곱셈 연산자를 이용하여 구현한다.

· **검증 방법**

시뮬레이션을 이용하여 다음 값을 검증하였다.

A	B	결과
3	2	6
5	4	20
15	15	225

· **핵심코드**

- mult_4bit (최상위 모듈)

```
`timescale 1ns/1ps
module mult_4bit(input wire [3:0] a, b, output wire [7:0] m);
    multiply_unsigned #(.WIDTH(4)) u_multiply(a, b, m);
endmodule
```

최상위 모듈은 FPGA 보드의 입출력과 실제 곱셈 모듈을 연결하는 역할을 수행한다. 입력 신호 a, b와 출력 신호 m을 정의하고, 내부에서 multiply_unsigned 모듈을 인스턴스화하여 사용한다. 이 과정에서 WIDTH 값을 4로 설정하여 4비트 곱셈기로 동작하도록 하였다. 최상위 모듈을 사용함으로써 곱셈 기능과 입출력 연결 기능을 분리할 수 있으며, 모듈 구조를 체계적으로 구성할 수 있다.

- multiply_unsigned (설계 모듈)

```
`timescale 1ns/1ps
// 4비트 곱셈: 입력 두 개를 곱해 8비트 결과를 만듭니다.
module multiply_unsigned #(parameter WIDTH = 4)(
    input wire [WIDTH-1:0] a, b,
    output wire [2*WIDTH-1:0] product
);
    assign product = a * b;
endmodule
```

최상위 모듈은 FPGA 보드의 입출력과 실제 곱셈 모듈을 연결하는 역할을 수행한다. 입력 신호 a, b와 출력 신호 m을 정의하고, 내부에서 multiply_unsigned 모듈을 인스턴스화하여 사용한다. 이 과정에서 WIDTH 값을 4로 설정하여 4비트 곱셈기로 동작하도록 하였다. 최상위 모듈을 사용함으로써 곱셈 기능과 입출력 연결 기능을 분리할 수 있으며, 모듈 구조를 체계적으로 구성할 수 있다.

- TB

```
`timescale 1ns/1ps
module tb;
    reg [3:0] a, b;
    wire [7:0] m;
    mult_4bit dut(a, b, m);
    integer x, y;
    initial begin
        for (x=0; x<16; x=x+1)
            for (y=0; y<16; y=y+1) begin
                a=x; b=y; #1;
                if (m !== x*y) $fatal(1, "multiply a=%0d b=%0d m=%0d", x,y,m);
            end
        $display("PASS multiplier: 256 input pairs"); $finish;
    end
    initial begin #1000; $fatal(1, "timeout"); end
endmodule
```

TB에서는 mult_4bit dut(a, b, m); 구문을 통해 검증 대상 회로(DUT, Device Under Test)를 생성하였다. 입력값은 reg 타입의 a와 b를 사용하여 TB 내부에서 자유롭게 변경할 수 있도록 하였고, 출력값 m은 회로의 결과를 받아오기 위해 wire 타입으로 선언하였다. 이를 통해 실제 FPGA 회로와 동일한 환경에서 시뮬레이션 검증을 수행할 수 있다. 또한, 이중 for문을 사용하여 두 입력에 들어올 수 있는 모든 값을 자동으로 생성하였다. 4비트 입력은 각각 0부터 15까지 총 16개의 값을 가질 수 있으므로, 모든 경우의 수는 16×16=25616 \times 16 = 25616×16=256개이다. 이를 통해 특정 입력만 확인하는 것이 아니라 가능한 모든 입력 조합에 대해 곱셈기가 올바르게 동작하는지 체계적으로 검증할 수 있다.

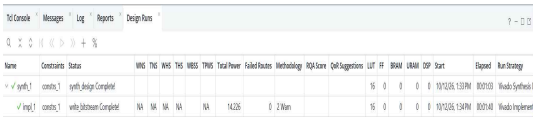
· **시뮬레이션 자료**

- Bit Hash

```
PS D:\Engineering\Vivado_Work\Lab4\mult_4bit> Get-FileHash .\mult_4bit.runs\impl_1\mult_4bit.bit -Algorithm SHA256 | Format-List Algorithm,Hash,Path


Algorithm : SHA256
Hash      : E68E42436078A578629A6F79891057AB34EE6B44767FB9982DBA89816ACBD292
Path      : D:\Engineering\Vivado_Work\Lab4\mult_4bit\mult_4bit.runs\impl_1\mult_4bit.bit
```

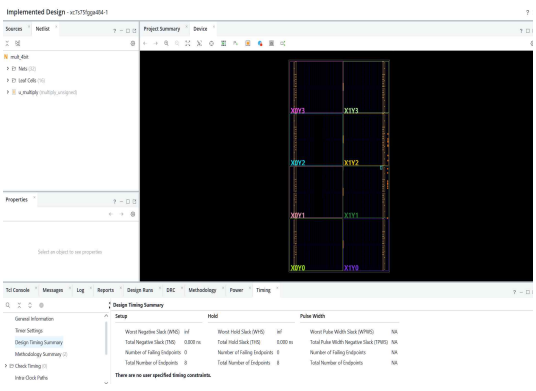
- Bitstream



- DRC



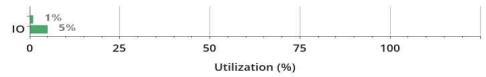
- Timing



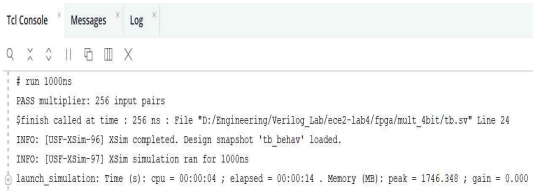
- Utilization

Summary

Resource	Utilization	Available	Utilization %
LUT	16	48000	0.03
IO	16	338	4.73



- Vivado Pass



- Simulation



· **검증 결과**

모든 입력 조합에 대해 예상한 곱셈 결과가 출력되는 것을 확인하였다. FPGA 보드에서도 스위치 입력에 따라 LED 출력이 정상적으로 변화하였다.

4. FPGA 실습 2 : BCD-이진수 변환

· **설계 목표**

BCD(Binary Coded Decimal) 값을 이진수로 변환하고, 변환 결과를 FPGA에서 확인한다.

· **설계 방법**

BCD 숫자는 각 자리수를 '0000 ~ 1001' 형태로 표현한다.

ex.) BCD : 0011 0101
- 10진수 : 35
- 이진수 : 100011

변환 연산 : binary = tens * 10 + ones;

· **검증 방법**

BCD 입력	10진수	이진수 출력
12	12	0001100
35	35	0100011
99	99	1100011

예상 결과와 실제 결과가 일치하는 것을 확인하였다.

· **핵심 코드**

- 설계 모듈 (bcd_to_binary)

```
`timescale 1ns/1ps
// 십의 자리와 일의 자리를 각각 0~9로 입력합니다.
module bcd_to_binary(
    input wire [3:0] ten, one,
    output wire [6:0] binary,
    output wire valid
);
    assign binary = ten * 7'd10 + one;
    assign valid = (ten <= 9) && (one <= 9);
endmodule
```

BCD(Binary Coded Decimal) 형식으로 입력된 십의 자리(ten)와 일의 자리(one)를 일반 이진수로 변환하는 역할을 수행한다. 입력은 각각 4비트로 구성되며, 출력은 최대 값 99를 표현할 수 있도록 7비트(binary)로 선언되었다. assign binary = ten * 7'd10 + one; 문장을 이용하여 십의 자리에 10을 곱한 후 일의 자리를 더함으로써 BCD 값을 이진수 형태의 숫자로 변환한다. 또한 assign valid = (ten <= 9) && (one <= 9); 를 통해 두 입력이 모두 0~9 범위에 존재하는 정상적인 BCD 값인지 검사하며, 조건을 만족할 경우 valid 신호를 출력하여 유효한 데이터임을 표시한다.

- 설계 모듈 (최상위 연결)

```
`timescale 1ns/1ps
module bcd_conv(
    input wire [3:0] one, ten,
    output wire [6:0] bin,
    output wire valid
);
    bcd_to_binary u_convert(ten, one, bin, valid);
endmodule
```

최상위 모듈은 FPGA 보드의 입출력 신호와 실제 변환 모듈을 연결하는 역할을 담당한다. 입력으로 십의 자리(ten)와 일의 자리(one)를 받아 bcd_to_binary 모듈에 전달하고, 변환 결과(bin)와 유효성 검사 결과(valid)를 외부로 출력한다. 실제 변환 연산은 내부의 bcd_to_binary 모듈에서 수행되며, 최상위 모듈은 각 입출력 신호를 연결하는 인터페이스 역할만 수행한다. 이러한 계층적 설계 구조를 통해 기능 모듈의 재사용성을 높이고 회로 설계를 체계적으로 관리할 수 있다.

- TB

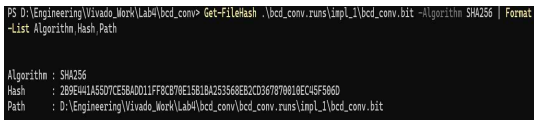
```
`timescale 1ns/1ps
module tb;
    reg [3:0] ten, one;
    wire [6:0] bin;
    wire valid;
    bcd_conv dut(one, ten, bin, valid);
    integer t, o;
    initial begin
        for(t=0;t<16;t=t+1)
            for(o=0;o<16;o=o+1) begin
                ten=t; one=o; #1;
                if(valid !== ((t<10)&&(o<10))) $fatal(1,"BCD valid");
                if(t<10 && o<10 && bin !== t*10+o) $fatal(1,"BCD value");
            end
        $display("PASS BCD: 100 valid and 156 invalid inputs"); $finish;
    end
    initial begin #1000; $fatal(1,"timeout"); end
endmodule
```

TB는 설계한 BCD-이진수 변환 회로의 동작을 검증하기 위한 시뮬레이션 환경이다. 두 개의 중첩된 for 문을 사용하여 십의 자리와 일의 자리에 가능한 모든 4비트 입력값(0~15)을 적용하며, 총 256개의 입력 조합에 대해 회로를 자동으로 검증한다. 입력값이 0~9 범위에 있을 경우 valid 신호가 1로 출력되는지 확인하고, 동시에 변환된 이진수

출력(bin)이 실제 계산값인 (십의 자리 × 10 + 일의 자리)와 일치하는지를 검사한다. 만약 결과가 다를 경우 $fatal 명령으로 즉시 시뮬레이션을 종료하여 오류를 보고하며, 모든 검증을 통과하면 PASS 메시지를 출력하고 시뮬레이션을 종료한다. 또한 타임아웃 기능을 추가하여 예상치 못한 설계 오류로 시뮬레이션이 무한정 실행되는 상황을 방지하였다.

· **시뮬레이션 자료**

- Bit Hash



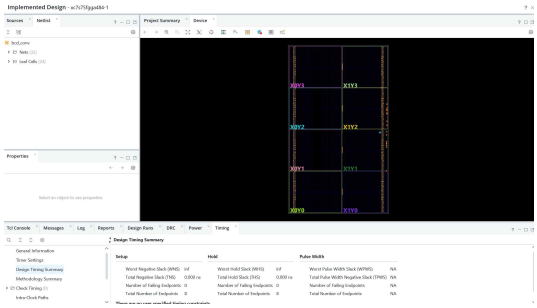
- Bitstream



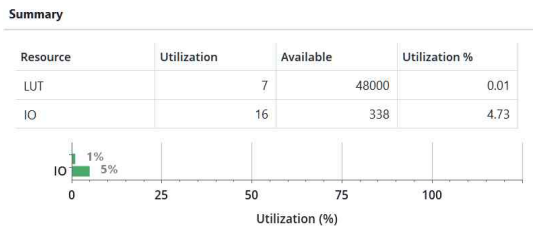
- DRC



- Timing



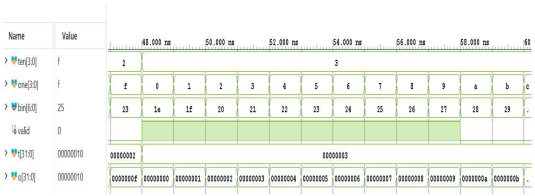
- Utilization



- Vivado Pass



- Simulation



· **검증 결과**

십의 자리와 일의 자리에 입력 가능한 모든 4비트 조합(총 256개)에 대해 검증을 수행하였다. 정상적인 BCD 입력(00~99)의 경우 valid 신호가 활성화되고 출력된 이진수 값이 입력된 십진수 값과 정확히 일치함을 확인하였다. 또한 BCD 규격을 벗어나는 입력(각 자릿수가 10~15인 경우)에 대해서는 valid 신호가 비활성화되어 비정상 입력을 정상적으로 검출하였다. 이를 통해 설계한 BCD-이진수 변환 회로가 정확하게 동작함을 확인하였다.

5. 최종 고찰

본 실험에서는 STM32와 FPGA를 활용하여 임베디드 시스템과 디지털 회로 설계의 기초를 학습하였다. STM32 실습에서는 버튼 입력을 읽어 LED를 제어하는 프로그램을 작성하면서 GPIO 입출력 제어와 폴링 방식의 동작 원리를 이해할 수 있었다. 특히 버튼을 누를 때마다 LED 점멸 간격이 변경되도록 구현하고, 상승 에지 검출과 디바운싱 기법을 적용하여 실제 스위치 입력 처리 과정에서 발생할 수 있는 문제를 해결할 수 있었다. 또한 동일한 기능을 외부 LED에 적용하면서 소프트웨어와 하드웨어의 연동 과정을 확인하였다.

FPGA 실습에서는 Verilog HDL을 이용하여 4비트 곱셈기와 BCD-이진수 변환 회로를 설계하였다. 모듈 단위로 회로를 설계하고 테스트벤치를 작성하여 모든 입력 조합에 대해 동작을 검증함으로써 체계적인 하드웨어 설계 및 검증 과정을 경험할 수 있었다. 특히 입력의 비트 수에 따라 출력 비트 폭이 결정된다는 점과, 정상 및 비정상 입력을 구분하는 검증 로직의 중요성을 확인할 수 있었다.

이번 실험을 통해 STM32는 프로그램의 순차적 실행을 통해 동작을 구현하고, FPGA는 하드웨어 구조 자체를 설계하여 병렬적으로 동작한다는 차이를 이해할 수 있었다. 또한 GPIO 제어, 상태 관리, 모듈화 설계, 테스트벤치 기반 검증 기법을 익히면서 임베디드 시스템과 디지털 회로 설계에 대한 기초 역량을 향상시킬 수 있었다. 이는 향후 더욱 복잡한 임베디드 및 FPGA 시스템을 설계하는 데 중요한 기반이 될 것으로 생각한다.

6. 참고문헌

[1] 서울시립대학교 전자전기컴퓨터공학부, STM32_0 : LED와 버튼 실습 교안, 2026.

[2] STMicroelectronics, STM32 Nucleo 64 Boards User Manual (UM1724), STMicroelectronics.

[3] Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, Prentice Hall, 2nd Edition, 2003.

7. Git-Hub 주소 및 최종 커밋

https://github.com/UnnamedDays/ECE-II-Project

e039c3f